

2. Introduction to Feature Extraction

Feature extraction reduces high-dimensional raw visual data into a compact, meaningful representation for algorithms, simplifying computational requirements while enhancing relevant information.

2.1 Types of Features and Techniques

- Local Features: Extracted from specific regions (keypoints, corners, patches) and used for object detection and image matching.
- Global Features: Computed over the entire image (color histograms, texture descriptors) to capture overall characteristics.
- Feature Extraction Techniques: Includes histogram-based methods (statistical distributions), filter-based methods (Gabor/Haar for edges/textures), and deep learning (CNNs that learn representations directly).
- Applications: Object recognition, face recognition, gesture recognition, medical imaging, and autonomous vehicle navigation. Challenges include variations in scale, lighting, noise, and orientation.

3. Histogram-Based Methods

3.1 HOG (Histogram of Oriented Gradients)

HOG captures local gradient and edge patterns, commonly leading to highly effective object and pedestrian detection. It is required because it focuses on shape and structural information rather than pixel intensity directly.

Working Mechanism:

1. Image Preprocessing: Convert to grayscale to simplify gradient calculation.
2. Gradient Computation: Calculate magnitude and orientation for each pixel, typically using a Sobel operator.
3. Cell Division: Divide the image into non-overlapping small cells (e.g., 8x8 pixels).
4. Histograms: Create a histogram of gradient orientations for each cell, where bins represent angle ranges and values represent frequency.
5. Block Normalization: Group cells into blocks and normalize the histograms to minimize lighting variations.
6. Feature Vector: Concatenate all block histograms to form the final HOG vector.

Python

```
from skimage.feature import hog
from skimage import exposure
import cv2
import matplotlib.pyplot as plt
```

```
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
```

```
features, hog_image = hog(image, pixels_per_cell=(8, 8), cells_per_block=(2, 2),
visualize=True)
hog_image_rescaled = exposure.rescale_intensity(hog_image, in_range=(0, 10))
```

```
plt.figure(figsize=(12, 6))
plt.subplot(121)
plt.imshow(image, cmap=plt.cm.gray)
plt.title('Original Image')
plt.subplot(122)
plt.imshow(hog_image_rescaled, cmap=plt.cm.gray)
plt.title('HOG Features')
plt.show()
```

```
print("HOG Feature Vector:\n", features)
```

3.2 Local Binary Pattern (LBP)

LBP is required for texture classification and face recognition because it robustly encodes local texture patterns by comparing a pixel to its neighbors.

Working Mechanism:

1. Image Preprocessing: Convert to grayscale.
2. Local Neighborhood Definition: Define a circular neighborhood based on a radius and a specific number of neighboring pixels.
3. Intensity Comparison: Compare each neighbor's intensity to the center pixel. Assign 1 if greater or equal; 0 otherwise.
4. Pattern Formation: Concatenate these binary values into a single LBP pattern.
5. Histogram Calculation: Count occurrences of each pattern across the image.
6. Feature Vector: The histogram acts as the feature vector.

Python

```
from skimage import feature
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
radius = 1
n_points = 8 * radius
lbp_image = feature.local_binary_pattern(image, n_points, radius, method='uniform')
```

```
hist, _ = np.histogram(lbp_image.ravel(), bins=np.arange(0, n_points + 3), range=(0, n_points + 2))
hist = hist.astype("float")
```

```
hist /= (hist.sum() + 1e-6)
```

```
plt.figure(figsize=(12, 6))  
plt.subplot(121), plt.imshow(lbp_image, cmap='gray'), plt.title('LBP Image')  
plt.subplot(122), plt.bar(range(0, n_points + 2), hist), plt.title('LBP Histogram')  
plt.show()  
print("LBP Feature Vector:\n", hist)
```

3.3 Histogram of Color

Required to capture the distribution of color intensities. It works by dividing the color space (RGB or HSV) into bins and counting the number of pixels in each bin.

3.4 Histogram of Edge Directions (HED)

HED captures the distribution of edge orientations, which leads to identifying dominant directions for texture analysis and object recognition.

Working Mechanism:

1. Image Preprocessing: Convert to grayscale; optionally apply Gaussian smoothing to reduce noise.
2. Edge Detection: Apply an edge algorithm (Canny, Sobel, Scharr) to highlight strong edges.
3. Orientation Calculation: Calculate the gradient angle for each edge pixel using arctangent.
4. Orientation Binning: Divide 360 degrees into bins (e.g., 8 octagonal directions) and assign pixel angles to them.
5. Histogram Formation: Count the pixels per bin.
6. Feature Vector: Use this histogram as the feature vector.

Python

```
import cv2  
import numpy as np  
import matplotlib.pyplot as plt  
  
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)  
image_smoothed = cv2.GaussianBlur(image, (5, 5), 0)  
edges = cv2.Canny(image_smoothed, threshold1=30, threshold2=70)  
  
gradient_x = cv2.Sobel(image_smoothed, cv2.CV_64F, 1, 0, ksize=3)  
gradient_y = cv2.Sobel(image_smoothed, cv2.CV_64F, 0, 1, ksize=3)  
gradient_orientation = np.arctan2(gradient_y, gradient_x) * 180 / np.pi  
  
hist, bin_edges = np.histogram(gradient_orientation, bins=8, range=(0, 360))
```

```
plt.figure(figsize=(12, 6))
plt.subplot(121), plt.imshow(edges, cmap='gray'), plt.title('Edge Map')
plt.subplot(122), plt.bar(bin_edges[:-1], hist, width=45, align='center')
plt.title('Histogram of Edge Directions')
plt.show()
print("HED Feature Vector:\n", hist)
```

3.5 Histogram of Intensity Gradients (HIG)

HIG captures gradient distributions directly from pixel intensities, which leads to robust texture and object classification.

Working Mechanism:

1. Preprocessing: Convert to grayscale.
2. Gradient Calculation: Compute dx and dy gradients (e.g., via Sobel).
3. Magnitude and Orientation: Compute $G = \sqrt{dx^2 + dy^2}$ and $\theta = \text{atan2}(dy, dx)$.
4. Histogram Construction: Bin the orientations (0 to 360 degrees) and increment based on pixel orientations.
5. Feature Extraction: Normalize the histogram to make it scale-invariant.
6. Feature Vector: Use the final histogram for computer vision tasks.

Python

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
dx = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
dy = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)

magnitude = np.sqrt(dx**2 + dy**2)
orientation = np.arctan2(dy, dx) * (180 / np.pi)

histogram, bins = np.histogram(orientation, bins=9, range=(0, 180))
histogram = histogram / histogram.sum()

plt.bar(bins[:-1], histogram, width=20)
plt.title("Histogram of Intensity Gradients (HIG)")
plt.show()
```

3.6 Texture Energy and Contrast Histograms

Required to quantify the physical variations of a surface. Energy measures uniformity (sum of squared intensities in a neighborhood), and contrast measures variation (standard deviation of intensities).

Working Mechanism:

1. Preprocessing: Convert to grayscale.
2. Texture Analysis: Define a local window size (e.g., 3x3).
3. Energy Calculation: Sum the squared values in the window.
4. Contrast Calculation: Find the standard deviation in the window.
5. Histogram Construction & Feature Extraction: Bin the values and extract them as feature vectors for texture classification.

Python

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
neighborhood_size = 3

energy = cv2.filter2D(image ** 2, -1, np.ones((neighborhood_size, neighborhood_size)))
contrast = cv2.filter2D(image, -1, np.ones((neighborhood_size, neighborhood_size)))

energy_histogram, energy_bins = np.histogram(energy, bins=256, range=(0, energy.max()))
contrast_histogram, contrast_bins = np.histogram(contrast, bins=256, range=(0,
contrast.max()))

energy_histogram = energy_histogram / energy_histogram.sum()
contrast_histogram = contrast_histogram / contrast_histogram.sum()

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1), plt.plot(energy_bins[:-1], energy_histogram, color='b'), plt.title("Texture
Energy")
plt.subplot(1, 2, 2), plt.plot(contrast_bins[:-1], contrast_histogram, color='r'), plt.title("Texture
Contrast")
plt.show()
```

Task: Texture Analysis for Material Classification

For classifying wood, metal, and fabric, two primary techniques are applicable:

1. Local Binary Pattern (LBP): Works by thresholding circular neighborhoods to generate binary patterns. Suitable for fabric weaves or wood grains. *Advantage*: Computationally efficient and resistant to lighting shifts. *Limitation*: Struggles with large macro-textures.

2. Texture Energy/Contrast: Works by measuring local variance and squared intensity sums. Suitable for distinguishing between polished metal (low contrast) and rough wood (high contrast). *Advantage:* Directly measures physical surface variance. *Limitation:* Highly susceptible to noise and shadowing.

4. Filtering & Convolution

Filtering is required for noise reduction, feature extraction, and edge detection. It leads to modified pixel values by sliding a matrix (filter/kernel) over an image and summing the dot products.

Specific Filtering Examples:

Python

```
import cv2
```

```
import numpy as np
```

```
image = cv2.imread('image.jpg')
```

```
# 1. Box Blur (noise reduction via averaging)
```

```
kernel_box = np.ones((3, 3), dtype=np.float32) / 9
```

```
blurred = cv2.filter2D(image, -1, kernel_box)
```

```
# 2. Gaussian Blur (smoother noise reduction)
```

```
kernel_gauss = cv2.getGaussianKernel(5, 1.0)
```

```
gaussian_blur = cv2.filter2D(image, -1, kernel_gauss)
```

```
# 3. Edge Detection (Sobel/Scharr for rapid intensity changes)
```

```
sobel_x = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
```

```
scharr_x = cv2.Scharr(image, cv2.CV_64F, 1, 0)
```

```
# 4. Embossing (3D stylized effect)
```

```
kernel_emboss = np.array([[-2, -1, 0], [-1, 1, 1], [0, 1, 2]], dtype=np.float32)
```

```
embossed_image = cv2.filter2D(image, -1, kernel_emboss)
```

4.2.1 Convolution Types

Convolution padding rules dictate the spatial size of the resulting feature map.

1. Standard Convolution: Full processing with default implicit padding.
2. Valid Convolution (No Padding): Kernel only processes pixels where it fully overlaps, shrinking the output dimensions.
3. Same Convolution (Zero Padding): Adds padding to ensure the output matches the input dimensions.
4. Valid Convolution with Strides: Skips pixels based on a stride value, significantly reducing output size.

Python

```
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
kernel = np.array([[1, 0, -1], [2, 0, -2], [1, 0, -1]], dtype=np.float32)
```

Standard

```
res_std = cv2.filter2D(image, -1, kernel)
```

Valid

```
res_valid = cv2.filter2D(image, -1, kernel, borderType=cv2.BORDER_CONSTANT)
```

Same

```
res_same = cv2.filter2D(image, -1, kernel, borderType=cv2.BORDER_REFLECT)
```

Valid with Strides

Note: In pure OpenCV, filter2D doesn't handle strides natively in a single call this way,
but the manual conceptually demonstrates it as:

```
res_stride = cv2.filter2D(image, -1, kernel, borderType=cv2.BORDER_CONSTANT) # (Manual  
syntax simulation)
```

5. Edge Detection & Common Techniques

Edge detection is a required preprocessing step to identify boundaries and reduce data size while highlighting structural features[cite: 46].

5.1 Common Techniques

- Gradient-Based: Calculates rate of change. Includes Sobel, Prewitt, and Scharr.
- Canny: Multi-stage algorithm known for noise reduction and accuracy.
- Laplacian of Gaussian (LoG): Enhances edges via zero-crossings in the second derivative.
- Marr-Hildreth: Uses LoG after Gaussian smoothing.
- Sobel-Feldman: Emphasizes diagonal edges alongside vertical/horizontal.
- CNNs: Deep learning applied to complex edge pattern extraction.

Sobel Implementation:

Python

```
import cv2
import numpy as np
```

```
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
gradient_x = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
gradient_y = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)
```

```
gradient_magnitude = np.sqrt(gradient_x**2 + gradient_y**2)
```

```
edges = (gradient_magnitude > 100).astype(np.uint8) * 255
```

5.2 The Canny Edge Detector

Required for producing thin, highly accurate continuous edge lines while strictly suppressing noise.

Key Steps & Working Mechanism:

1. Gaussian Smoothing: Blurs the image to prevent false edges from noise.
2. Gradient Calculation: Uses Sobel to find the intensity rate of change direction and magnitude.
3. Non-Maximum Suppression: Thins edges by only keeping the local maxima along the gradient line.
4. Edge Tracking by Hysteresis: Links continuous lines using high and low thresholds.

Parameters:

- Gaussian Kernel Size: A larger size creates a smoother image but risks losing fine detail.
- High Threshold (T_{high}): Defines definitive strong edge points.
- Low Threshold (T_{low}): Minimum requirement; rejects anything below it. Pixels between low and high are connected to strong edges if adjacent.

Python

```
import cv2
```

```
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)  
edges = cv2.Canny(image, threshold1=100, threshold2=200)
```

```
cv2.imshow('Canny Edges', edges)  
cv2.waitKey(0)  
cv2.destroyAllWindows()
```